Threat-Intelligence Kill-Chain Correlation Engine

Project Report submitted for the partial fulfilment of grade for the course

Data Structures and Algorithms -3 (25CS2103E)

in

A.Y. 2026-2027

Submitted by
V. Hasith Varma (2520090125)
J. Amrutha Varshini (2520080066)

Team Number: 8

Section: 12

Under the guidance of

Ms. K. Chandusha

Asst. Prof., CS&IT



Hyderabad-500090, Telangana, India.

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



DECLARATION

The Mini Project Report entitled "**Threat-Intelligence Kill-Chain Correlation Engine"** is a record of Bonafide work of **V. Hasith Varma – 2520090125, J. Amrutha Varshini – 2520080066** submitted in partial fulfillment for the award of B. Tech in Computer Science and Information Technology to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

V. Hasith Varma – 2520090125

J. Amrutha Varshini – 2520080066

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



CERTIFICATE

This is to certify that the mini project based report entitled "**Threat-Intelligence Kill-Chain Correlation Engine"** is a bonafide work done and submitted by **V. Hasith Varma – 2520090125, J. Amrutha Varshini – 2520080066** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science and Information Technology, K L (Deemed to be University), during the academic year **2026-2027.**

Signature of the Supervisor

Signature of the HOD **Signature of the External Examiner**

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to our guide, Ms. K. Chandusha, Assistant Professor, Department of Computer Science and Information Technology, which cannot be expressed in words for her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. Khaja Shareef (CSIT)**, Head of the *Department for Computer Science and Information Technology* for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

V. Hasith Varma – 2520090125

J. Amrutha Varshini – 2520080066

TABLE OF CONTENTS

S.No.	Contents	Page no.
1.	Abstract	6
2.	Introduction	7
3.	Literature survey	8
4.	Problem Statement and Objectives	10
5.	Hardware and Software requirements	11
6.	Implementation	12
7.	Results & Discussion	16
8.	Conclusion & Future Scope	22
9.	References	23

1.ABSTRACT

The Threat-Intelligence Kill-Chain Correlation Engine is developed to help Security Operations Centres (SOCs) analyse large amounts of security logs and alerts more efficiently. Instead of manually checking thousands of log entries, the system automatically identifies known attack patterns and connects related events to recreate the complete sequence of a cyberattack, known as a kill chain.

It represents attack sequences as graph structures to identify critical choke points where defensive actions can effectively interrupt the attack. In addition, the system recommends a minimal set of monitoring points required to cover all possible attack paths and assigns detected alerts to security analysts based on priority. This helps security teams understand the progression of an attack more clearly and focus their attention on the most important security events.

By applying Data Structures and Algorithms such as hashing, graph traversal, pattern matching, and graph optimization techniques, this project demonstrates how DSA concepts can solve real-world cybersecurity challenges while improving the speed and accuracy of threat detection and analysis.

Team Name: Team 8

Team Logo (if any): Not applicable

Team Members:

1. V. Hasith Varma, 2520090125
2. J. Amrutha Varshini, 2520080066

2. INTRODUCTION

A Security Operations Centre (SOC) receives an enormous number of security logs and alerts every day from firewalls, servers, network sensors and endpoints. Examining every event manually is slow, and a single alert rarely shows the complete picture of an attack. Real attacks unfold in stages, commonly described by the Cyber Kill Chain model: reconnaissance, weaponization, delivery, exploitation, installation, command and control, and actions on objectives [2]. When these stages appear as separate, unrelated alerts, analysts must link them by hand, which delays detection.

The Threat-Intelligence Kill-Chain Correlation Engine is built to address this gap. It processes security logs, detects known suspicious patterns, converts them into alerts, and correlates related alerts to reconstruct the attack as a sequence of stages, showing the analyst what happened, how the attack progressed, and which events deserve attention first.

The project is an application of Data Structures and Algorithms to a practical problem. Pattern matching searches the log content for known attack indicators, and a hash table gives constant-time lookup of attack signatures. Related alerts are stored as an attack graph using an adjacency list, which is explored with Breadth-First Search (BFS) and Depth-First Search (DFS). Articulation points identify critical choke points, a greedy graph-optimization approach selects monitoring points that cover the attack paths, and a priority queue (max heap) ensures that high-priority alerts are handled before low-priority ones.

The system is delivered as a web application: the frontend presents logs, alerts, kill chains, attack graphs and monitoring information; the backend runs the processing pipeline and APIs; and MongoDB stores the processed data.

The rest of this report reviews similar work (Section 3), states the problem and objectives (4), lists the requirements (5), presents the implementation (6), discusses the results (7), and gives the conclusion, future scope and references (8–9).

3. LITERATURE SURVEY

The following existing tools, frameworks and projects were studied. Each deals with one or more parts of the problem addressed by this project.

1. **MITRE ATT&CK [1]** is a globally used knowledge base of adversary tactics and techniques built from real-world observations. SOC teams use it to map detections to attacker behaviour and to describe attacks in a common vocabulary. It is a reference catalogue rather than a tool that correlates an organisation's own alerts, but its staged view of attacker behaviour supports the ordering of events into attack sequences in our project.

2. **Lockheed Martin Cyber Kill Chain [2]** is an intelligence-driven defence model that breaks an intrusion into seven stages, from reconnaissance to actions on objectives. It shows why interrupting any one stage can stop an attack. The model is conceptual and does not process logs by itself; our engine operationalises it by reconstructing the kill chain automatically from detected alerts.

3. **Wazuh [3]** is an open-source security platform that provides log analysis, intrusion detection and file-integrity monitoring using rule-based detection. It collects logs from many hosts and raises an alert when a rule matches. Alerts are mainly produced rule by rule, so analysts still have to connect them manually to understand how an attack progressed, which is the step our engine automates.

4. **Suricata [4]** is an open-source network intrusion detection and prevention engine that inspects traffic against a large set of signature rules using efficient multi-pattern matching, a technique that traces back to the Aho–Corasick algorithm [9]. It demonstrates the value of fast signature-based detection. It focuses on detecting individual events and does not rebuild the sequence of an attack across hosts.

5. **Sigma [5]** is an open, vendor-neutral rule format for describing suspicious log events, so that one detection rule can be reused across different SIEM products. It confirms the usefulness of a shared signature database for log analysis. Its focus is the detection rule itself, not the graph-based correlation, critical-point analysis or monitoring optimisation performed in this project.

6. **BloodHound [6]** applies graph theory to reveal hidden relationships and attack paths in Active Directory environments, helping defenders find the shortest routes an attacker could take. It proves

that graph models are effective for attack-path analysis. It targets privilege relationships inside a directory service rather than correlating security-log alerts into kill chains.

7. **Splunk Enterprise Security [7]** is a commercial SIEM that collects machine data, runs correlation searches and uses risk-based alerting to rank notable events for analysts. It shows the importance of alert correlation and prioritisation in a SOC. It is a large, licensed platform, whereas our project is a lightweight implementation that makes the underlying data structures and algorithms explicit.

8. **Elastic Security [8]** combines SIEM and endpoint protection on top of Elasticsearch, offering prebuilt detection rules mapped to MITRE ATT&CK and investigation views for tracing related events. It shows how detection and investigation can be brought together in one interface. Its correlation depends on rules and analyst investigation, while our engine builds the attack graph and derives choke points and monitoring points from it.

From this survey it is clear that existing solutions are strong at either detecting individual events, cataloguing attacker behaviour, or analysing graphs in a specific domain. Graph algorithms such as articulation-point detection [10] are well established but are rarely applied directly to alert correlation. This project combines pattern-based detection, hashing, kill-chain correlation, attack-graph analysis, monitoring optimisation and alert prioritisation in a single, understandable pipeline.

4. Problem Statement and Objectives

Problem Statement:

Security analysts find it difficult to manually analyse large volumes of logs and isolated alerts, which makes it hard to see how an attack progresses and which alerts to handle first.

Problem Description:

Individual alerts rarely show the complete picture of an attack. An automated system is needed to detect known attack patterns, correlate related alerts into kill chains, and highlight critical points and high-priority alerts.

Objectives:

Identify the Project's Purpose:

The project was initiated to reduce the manual effort of analysing security data in a SOC by connecting isolated alerts into meaningful attack sequences, and to show how Data Structures and Algorithms can solve this real-world cybersecurity problem. The objectives of the project are:

- To detect known attack patterns in security logs using pattern matching, with hashing for fast attack-signature lookup.
- To generate security alerts from detected patterns and correlate related alerts and events into attack sequences (kill chains).
- To represent attack activity as a graph (adjacency list) and analyse attack paths using BFS and DFS.
- To identify critical choke points in the attack graph using articulation points, where defensive action can interrupt the attack.
- To recommend a small set of monitoring points that covers the attack paths, using a greedy graph-optimization approach.
- To prioritise alerts using a priority queue (max heap) so that important alerts are handled first, and to present all results to the analyst through a web dashboard.

5. HARDWARE AND SOFTWARE REQUIREMENTS

Hardware Requirements:

- Processor: Intel Core i5 (or equivalent) or higher
- RAM: 8 GB minimum (16 GB recommended)
- Storage: 256 GB SSD or higher with at least 10 GB free space

Software Requirements:

- Operating System: Windows 10/11 (Linux or macOS also supported)
- Frontend: Web application (HTML, CSS, JavaScript) running in a modern web browser
- Backend: Server-side application exposing REST APIs for the processing pipeline and graph analysis (Node.js)
- Database: MongoDB for storing logs, alerts, correlations, kill chains, graphs and analysis results

6. IMPLEMENTATION

Module 1: Pattern Matching, Signature Lookup (Hashing) and Alert Generation

Each log line is scanned with the KMP pattern-matching algorithm for known suspicious patterns. A matched pattern is resolved in O(1) time through a hash table of attack signatures, and an alert is generated.

```
// Hash table of known attack signatures (pattern -> signature details)
const signatures = new Map([
  ["nmap scan", { id: "SIG-001", name: "Port Scan",
                  stage: "Reconnaissance", severity: 3 }],
  ["failed password", { id: "SIG-002", name: "SSH Brute Force",
                        stage: "Delivery", severity: 6 }],
  ["union select", { id: "SIG-003", name: "SQL Injection",
                     stage: "Exploitation", severity: 8 }],
  ["powershell -enc", { id: "SIG-004", name: "Encoded PowerShell",
                        stage: "Installation", severity: 8 }],
  ["beacon", { id: "SIG-005", name: "C2 Beaconing",
               stage: "Command & Control", severity: 9 }],
]);

// KMP: longest-proper-prefix table
function buildLPS(p) {
  const lps = new Array(p.length).fill(0);
  let len = 0, i = 1;
  while (i < p.length) {
    if (p[i] === p[len]) lps[i++] = ++len;
    else if (len) len = lps[len - 1];
    else lps[i++] = 0;
  }
  return lps;
}

// KMP search: true if pattern occurs in text, O(n + m)
function kmpSearch(text, pattern) {
  const lps = buildLPS(pattern);
  let i = 0, j = 0;
  while (i < text.length) {
    if (text[i] === pattern[j]) {
      i++; j++;
      if (j === pattern.length) return true;
    } else if (j) j = lps[j - 1];
    else i++;
  }
  return false;
}

// Scan logs, look up each matched pattern in the hash table, create alerts
function detectAlerts(logs) {
  const alerts = [];
  for (const log of logs) {
    const line = log.message.toLowerCase();
    for (const pattern of signatures.keys()) {
      if (kmpSearch(line, pattern)) {
        const sig = signatures.get(pattern);          // O(1) hash lookup
        alerts.push({
          signatureId: sig.id, name: sig.name, stage: sig.stage,
          severity: sig.severity, src: log.srcIp, dst: log.dstHost,
          time: log.timestamp,
        });
```

```
      }
    }
  }
  return alerts;
}
```

Module 2: Alert Correlation, Attack Graph (Adjacency List), BFS and DFS

Related alerts are correlated into an attack graph stored as an adjacency list. Alerts are ordered by kill-chain stage, BFS explores connected nodes level by level, and DFS enumerates the attack paths between two nodes.

```
const STAGES = ["Reconnaissance", "Weaponization", "Delivery", "Exploitation",
                "Installation", "Command & Control", "Actions on Objectives"];

class AttackGraph {
  constructor() { this.adj = new Map(); }           // node -> [{ to, alerts }]

  addNode(n) { if (!this.adj.has(n)) this.adj.set(n, []); }

  addEdge(u, v, alert) {                            // attacker/host -> host
    this.addNode(u); this.addNode(v);
    const edge = this.adj.get(u).find((e) => e.to === v);
    if (edge) edge.alerts.push(alert);              // merge repeated alerts
    else this.adj.get(u).push({ to: v, alerts: [alert] });
  }

  // BFS: explore connected attack nodes level by level
  bfs(start) {
    const seen = new Set([start]), order = [], queue = [start];
    while (queue.length) {
      const u = queue.shift();
      order.push(u);
      for (const { to } of this.adj.get(u) || [])
        if (!seen.has(to)) { seen.add(to); queue.push(to); }
    }
    return order;
  }

  // DFS: enumerate every simple attack path from src to dst
  dfsPaths(src, dst) {
    const paths = [], path = [src], visited = new Set([src]);
    const go = (u) => {
      if (u === dst) { paths.push([...path]); return; }
      for (const { to } of this.adj.get(u) || []) {
        if (visited.has(to)) continue;
        visited.add(to); path.push(to);
        go(to);
        path.pop(); visited.delete(to);
      }
    };
    go(src);
    return paths;
  }
}

// Correlate alerts: every alert becomes an edge between its source and target
function buildAttackGraph(alerts) {
  const g = new AttackGraph();
  for (const a of alerts) g.addEdge(a.src, a.dst, a);
  return g;
}
```

```
// Reconstruct the kill chain: order alerts by stage, then by time
function reconstructKillChain(alerts) {
  return [...alerts]
    .sort((a, b) => STAGES.indexOf(a.stage) - STAGES.indexOf(b.stage)
                 || new Date(a.time) - new Date(b.time))
    .map((a) => a.stage + ": " + a.name + " (" + a.src + " -> " + a.dst + ")");
}
```

Module 3: Critical Points (Articulation Points) and Monitoring-Point Selection

Tarjan's DFS-based algorithm finds the articulation points, whose removal disconnects part of the attack path (choke points). A greedy strategy then picks the nodes that cover the most uncovered edges to form a small monitoring set.

```
// Undirected view of the attack graph (needed for articulation points)
function toUndirected(graph) {
  const und = new Map();
  const node = (n) => {
    if (!und.has(n)) und.set(n, new Set());
    return und.get(n);
  };
  for (const [u, list] of graph.adj)
    for (const { to } of list) { node(u).add(to); node(to).add(u); }
  return und;
}

// Tarjan's algorithm: articulation points in O(V + E)
function articulationPoints(und) {
  const disc = new Map(), low = new Map(), result = new Set();
  let timer = 0;

  const dfs = (u, parent) => {
    disc.set(u, ++timer); low.set(u, timer);
    let children = 0;
    for (const v of und.get(u)) {
      if (!disc.has(v)) {
        children++;
        dfs(v, u);
        low.set(u, Math.min(low.get(u), low.get(v)));
        if (parent !== null && low.get(v) >= disc.get(u)) result.add(u);
      } else if (v !== parent) {
        low.set(u, Math.min(low.get(u), disc.get(v)));
      }
    }
    if (parent === null && children > 1) result.add(u);   // root case
  };

  for (const u of und.keys()) if (!disc.has(u)) dfs(u, null);
  return [...result];
}

// Greedy monitoring selection: repeatedly pick the node covering the
// largest number of still-uncovered attack edges
function selectMonitoringPoints(und) {
  const key = (a, b) => (a < b ? a + "|" + b : b + "|" + a);
  const uncovered = new Set();
  for (const [u, nbrs] of und) for (const v of nbrs) uncovered.add(key(u, v));

  const chosen = [];
```

```
  while (uncovered.size) {
    let best = null, bestCount = 0;
    for (const [u, nbrs] of und) {
      let count = 0;
      for (const v of nbrs) if (uncovered.has(key(u, v))) count++;
      if (count > bestCount) { best = u; bestCount = count; }
    }
    chosen.push(best);
    for (const v of und.get(best)) uncovered.delete(key(best, v));
  }
  return chosen;
}
```

Module 4: Alert Prioritisation using a Priority Queue (Max Heap)

Alerts are inserted into a max heap keyed on a priority score (severity weighted with kill-chain stage), so the most important alert is always removed first in O(log n) time.

```
class MaxHeap {
  constructor() { this.h = []; }
  get size() { return this.h.length; }

  push(item) {                                          // O(log n)
    this.h.push(item);
    let i = this.h.length - 1;
    while (i > 0) {
      const p = (i - 1) >> 1;
      if (this.h[p].score >= this.h[i].score) break;
      [this.h[p], this.h[i]] = [this.h[i], this.h[p]];
      i = p;
    }
  }

  pop() {                                               // O(log n)
    if (!this.h.length) return null;
    const top = this.h[0], last = this.h.pop();
    if (this.h.length) {
      this.h[0] = last;
      let i = 0;
      const n = this.h.length;
      for (;;) {
        const l = 2 * i + 1, r = l + 1;
        let m = i;
        if (l < n && this.h[l].score > this.h[m].score) m = l;
        if (r < n && this.h[r].score > this.h[m].score) m = r;
        if (m === i) break;
        [this.h[m], this.h[i]] = [this.h[i], this.h[m]];
        i = m;
      }
    }
    return top;
  }
}

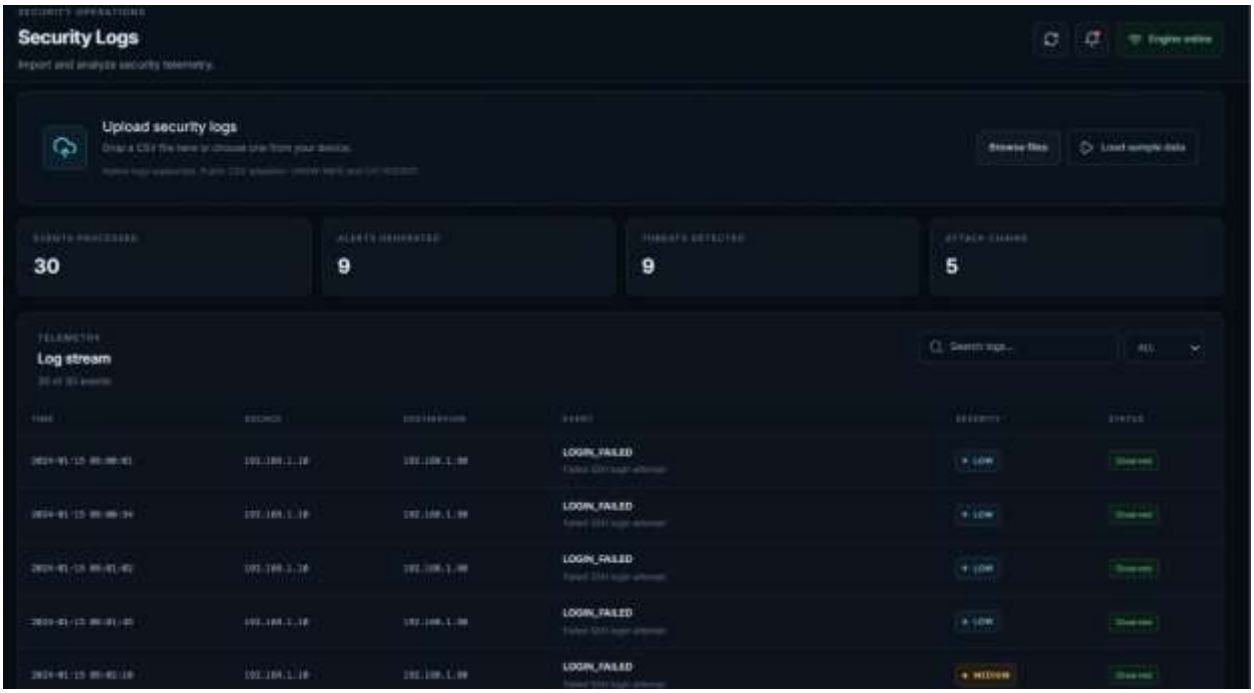
// Higher severity and later kill-chain stage => higher priority
function prioritiseAlerts(alerts) {
  const heap = new MaxHeap();
  for (const a of alerts)
    heap.push({ ...a, score: a.severity * 10 + STAGES.indexOf(a.stage) });

  const ordered = [];
  while (heap.size) ordered.push(heap.pop());
  return ordered;                                       // highest priority first
}
```

RESULTS & DISCUSSION

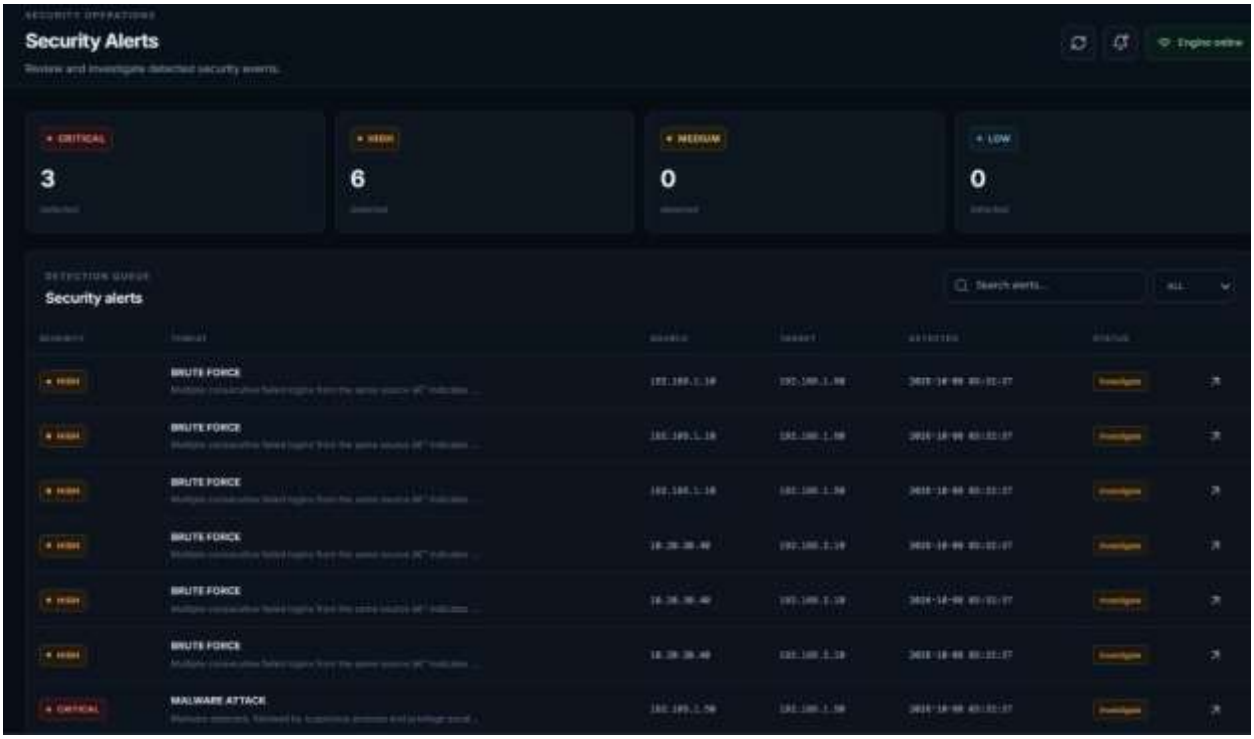
The application was run end to end on sample security logs. The main screens of the web application are shown below.

Figure 7.1: Security Logs and Dashboard



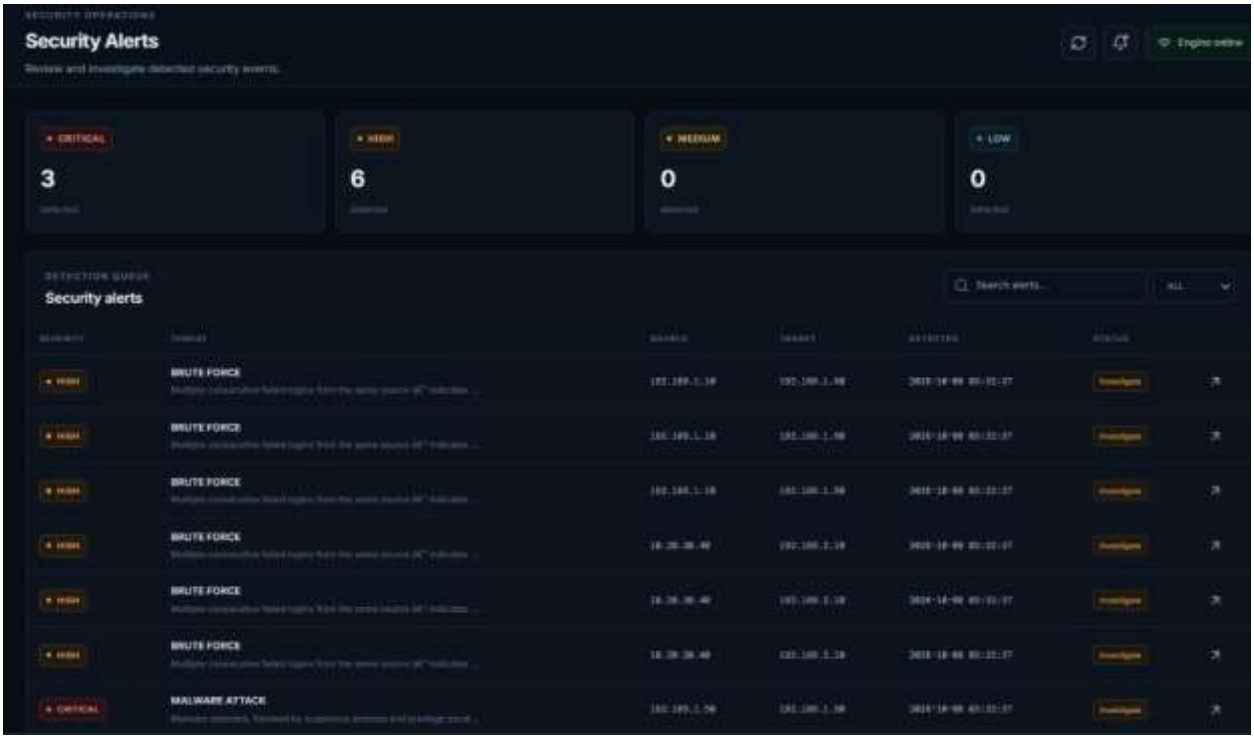
Security logs are ingested and processed through the pipeline, and the dashboard summarises the overall security activity for the analyst.

Figure 7.2: Alerts and Prioritisation



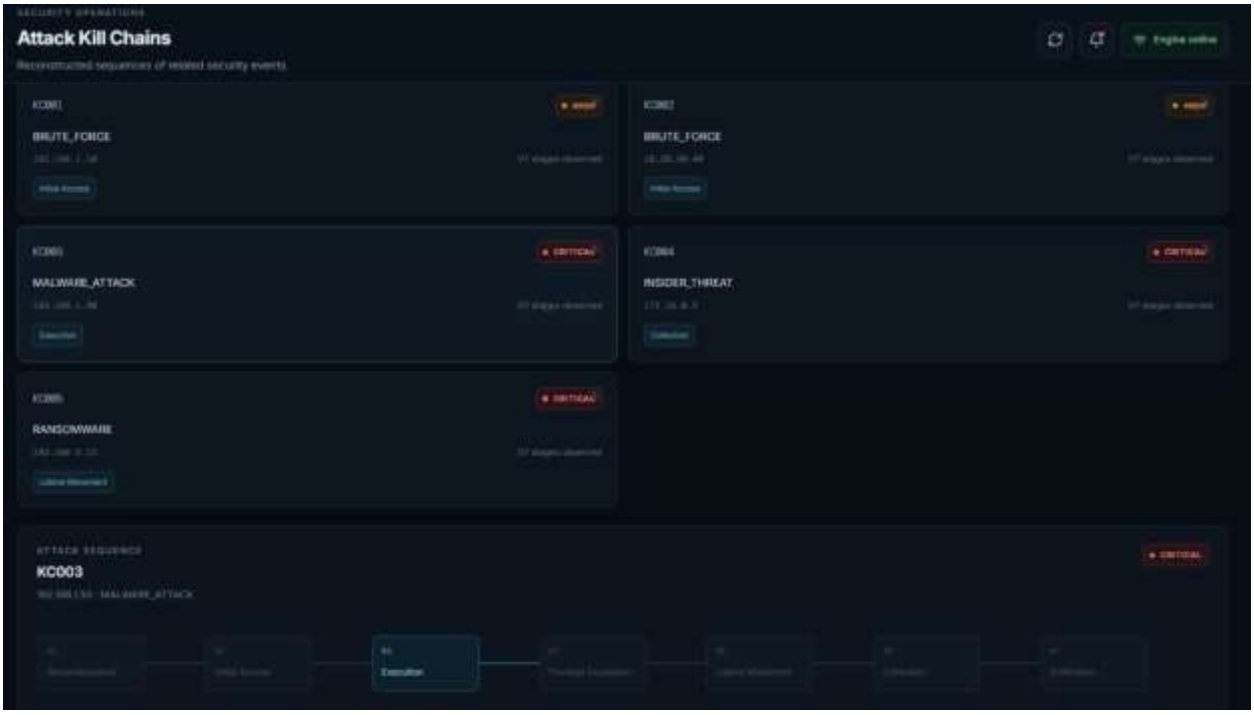
Detected patterns are converted into alerts, and the priority queue lists the higher-priority alerts ahead of the lower-priority ones.

Figure 7.3: Attack Investigation and Correlation



Related alerts and events are correlated, so the analyst can follow how one suspicious event connects to the next.

Figure 7.4: Kill Chain View



Correlated alerts are arranged as an ordered sequence of attack stages, giving a clear view of how the attack progressed.

Figure 7.5: Attack Graph and Critical Points



Attackers, hosts and events are shown as a graph, with BFS/DFS path analysis and the critical choke points (articulation points) highlighted.

Figure 7.6: Monitoring Coverage



The selected monitoring points that cover the attack paths are listed, helping the team decide where to place defensive monitoring.

Overall, the results show that the system turns raw security logs into alerts, attack chains and analysis outputs that can be read quickly by a security analyst.

7. CONCLUSION & FUTURE SCOPE

The project set out to help SOC analysts make sense of large volumes of security logs. The Threat-Intelligence Kill-Chain Correlation Engine meets this aim by detecting known attack patterns through pattern matching and hash-based signature lookup, generating alerts, and correlating related alerts into attack sequences. These sequences are represented as an adjacency-list graph and analysed with BFS and DFS, articulation points identify the critical choke points, a greedy approach selects monitoring points to cover the attack paths, and a max-heap priority queue brings the most important alerts to the front. The results are presented to the analyst through a web dashboard backed by MongoDB.

The key takeaway is that classical Data Structures and Algorithms, when combined in a well-designed pipeline, can reduce manual analysis effort and give security teams a clearer view of how an attack develops across different systems.

Future Scope: the project can be extended in the following directions.

Near term: extend the signature database and supported log formats, map detected alerts to MITRE ATT&CK techniques, and add filtering, search and export of kill-chain reports in the dashboard.

Medium term: support real-time streaming ingestion of logs, add anomaly-based detection (for example machine-learning models) alongside signature-based detection, and improve scalability with a dedicated graph database.

Long term: integrate live threat-intelligence feeds, automate response actions at the identified choke points through SOAR-style playbooks, and support distributed deployment across multiple organisations and large-scale networks.

8. REFERENCES

1. MITRE ATT&CK®, MITRE Corporation. https://attack.mitre.org/
2. Lockheed Martin, "The Cyber Kill Chain". https://www.lockheedmartin.com/en-us/capabilities/cyber/cyber-kill-chain.html
3. Wazuh – Open Source XDR and SIEM. https://wazuh.com/
4. Suricata – Network IDS, IPS and Network Security Monitoring Engine. https://suricata.io/
5. SigmaHQ – Generic Signature Format for SIEM Systems. https://github.com/SigmaHQ/sigma
6. BloodHound – Attack path analysis using graph theory. https://github.com/BloodHoundAD/BloodHound
7. Splunk Enterprise Security. https://www.splunk.com/en_us/products/enterprise-security.html
8. Elastic Security. https://www.elastic.co/security
9. A. V. Aho and M. J. Corasick, "Efficient string matching: an aid to bibliographic search," Communications of the ACM, vol. 18, no. 6, pp. 333–340, 1975.
10. R. Tarjan, "Depth-first search and linear graph algorithms," SIAM Journal on Computing, vol. 1, no. 2, pp. 146–160, 1972.